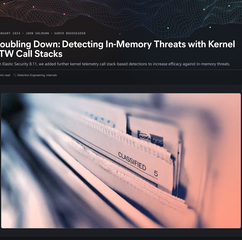
JANUARY 2024 · JON VALDMAN / GARTH BOUCHERON

Doubling Down: Detecting In-Memory Threats with Kernel ETW Call Stacks

With Elastic Security 8.11, we added further kernel telemetry call stack based detections to increase efficacy against in-memory threats.

5 min read · Threats Engineering, Malware



Introduction

We were pleased to see that the **kernel call stack** capability we released in 8.8 was met with **extremely positive community feedback**. - both from the offensive research teams attempting to evade us and the defensive teams hinging alerts future due to the additional **context**.

But this was only the first step. We needed to arm defenders with even more visibility from the kernel - the most reliable mechanism to combat user-mode threats. With the introduction of Kernel Patch Protection in 64-bit Windows, Microsoft created a shared responsibility model where security vendors are now offered only the kernel-level tracing and extension points that Microsoft provides. The most notable addition to this visibility is the **Microsoft-Windows-Threat-Intelligence Event Tracing for Windows**(ETW) API provider.

Microsoft has identified a handful of highly security-relevant syscalls and provided security vendors with near real time telemetry of those. While we would strongly prefer more granular telemetry that allow synchronous blocking of malicious activity, Microsoft has implicitly not deemed this a necessary security cost-yet. Currently the only blocking mechanism afforded to security vendors for these syscalls is user-mode hooking - and that approach is **inherently fragile**. In Elastic, we determined that a more robust detection approach based on kernel telemetry collected through ETW would provide product security value for most of our users while also empowering our customer SOCs that have the full fidelity of those events to opt into stepping up an advanced option available in Elastic policy and implement filtering tailored to their specific environments.

Currently, we propagate visibility into the following APs:

- `VirtualAlloc`
- `VirtualProtect`
- `MapViewOfFile`
- `VirtualAllocEx`
- `VirtualProtectEx`
- `MapViewOfFileEx`
- `SuspendThread` (call stacks not always available due to ETW limitations)
- `SetThreadContext` (call stacks planned for 8.12)
- `WriteProcessMemory`
- `MemPhysicalMemory` (basic) (planned for 8.12)

In addition to call stack information, our API events are also enriched with several **behaviors**:

API event	Description
`cross_process`	The observed activity was between two processes.
`native_api`	A call was made directly to the undocumented Native API rather than the supported Win32 API.
`direct_syscall`	A syscall instruction originated outside of the Native API layer.
`proxy_call`	The call stack appears to show a proxied API call to masking the true caller.
`execution_api`	Executable code-image memory is unexpectedly calling a sensitive API.
`injection`	Suspicious executable non-image memory is calling a sensitive API.
`image_hooked`	An entry in the call stack appears to have been hooked.
`image_callback_call`	An entry in the call stack was preceded by a call to a dynamically resolved function.
`image_rwx`	An entry in the call stack was not preceded by a call instruction.
`image_rx`	An entry in the call stack is writable.
`unbacked_rwx`	An entry in the call stack is non-image and writable.
`allocate_unbacked`	A region of non-image-executable memory suspiciously allocated more-executable memory.
`remote_flipmapping`	The PAGE_EXECUTE protection is unexpectedly fluctuating.
`write_flipmapping`	The PAGE_WRITE protection of executable memory is unexpectedly fluctuating.
`hook_api`	A change to the memory protection of a small executable-image memory region was observed.
`hollow_image`	A change to the memory protection of a large executable-image memory region was made.
`hook_unhooked`	A change to the memory protection was unexpectedly marked as PAGE_GUARD.
`hollow_unhooked`	A change to the memory protection of a large executable-image memory was made.
`guarded_code`	Executable memory was unexpectedly marked as PAGE_GUARD.
`hidden_code`	Executable memory was unexpectedly marked as PAGE_NOACCESS.
`remote_unbacked`	A region of non-image-executable memory was executed in an unexpected fashion.
`callback_unbacked_rwx`	A callback-backpatch was potentially set.

New Rules

8.8-11 - Elastic Defend's behavior protection comes with many new rules against various popular malware techniques, such as shellcode fluctuation, threadless injection, call stack spoofing, hollow calls, and AMSI or ETW patching.

These rules include:

Windows APi Call via Direct Syscall

Identifies the call of commonly abused Windows APIs to perform code injection and where the call stack is not starting with NTDLL.

```
api where event.category == "intrusion_detection" and

  process.Ext.api.behavior == "direct_syscall" and

  process.Ext.api.name == "VirtualAlloc" or "VirtualProtect",
                          "MapViewOfFile" or "WriteProcessMemory"
```

Windows APi Call via Direct Syscall rule logic

VirtualProtect via Random Indirect Syscall

Identifies calls to the VirtualProtect API and where the call stack is not originating from its equivalent NT syscall NtProtectVirtualMemory.

```
api where

  process.Ext.api.name == "VirtualProtect" and

  not _arraysearch(process.thread.Ext.call_stack, $entry, $entry.symbol_info == "ntdll.dll!NtProtectVirtualMemory", "ntdll.dll!ZwProtectVi
```

VirtualProtect via Random Indirect Syscall rule match examples

Image Hollow from Unbacked Memory

```
api where process.Ext.api.behaviors == "hollow_image" and

  process.Ext.api.name == "MapViewOfFile" and

  process.Ext.api.summary == "/c /Z27" and

  process.Ext.api.parameters.size == 10000 and process.executable != null and

  process.thread.Ext.call_stack_summary == "ntdll|kernel*

  process.Ext.api.summary == "/c /Z27" and

  process.Ext.api.parameters.size == 10000 and process.executable != null and

  process.thread.Ext.call_stack_summary == "ntdll|kernel*"
```

Image Hollow from Unbacked Memory rule match examples

Below example of matches on `msvcoe.dll` module attempting to replace it's memory content with a malicious payload.

AMSI and WLDP Memory Patching

Identifies attempts to modify the permissions or write to Microsoft Antimalware Scan Interface or the Windows Lock Down Policy related DLLs from memory to modify its behavior for evading malicious content checks.

```
api where

  (process.Ext.api.name : "VirtualProtect*" and
   process.Ext.api.parameters.protection : "*RW*") or

  process.Ext.api.name : "WriteProcessMemory*"
  and

  process.Ext.api.summary : ("*_AMSI.DLL*" "*_Wldp.dll*" "*_ntdll.dll*")
```

AMSI and WLDP Memory Patching rule match examples

Evasion via Event Tracing for Windows Patching

Identifies attempts to patch the Microsoft Event Tracing for Windows via memory modification.

```
api where process.Ext.api.name : "WriteProcessMemory*" and

  process.Ext.api.summary : ("*ntdll.dll!Etw*" "*ntdll.dll!Nt*Trace*") and

  not process.executable : ("?:\Windows\System32\*.exe" and "?:\Program Files\WindowsApps\*Microsoft.Windows.DevHomeOS.exe")
```

Evasion via Event Tracing for Windows Patching rule match examples

Windows System Module Remote Hooking

Identifies attempts to write to a remote process memory to modify NTDLL or KernelBase modules as a preparation step for stealthy code injection.

```
api where process.Ext.api.name : "WriteProcessMemory" and

  process.Ext.api.behaviors == "cross-process" and

  process.Ext.api.summary : ("*ntdll.dll*", "*KernelBase.dll*")
```

Below is an example of matches for `ThreadlessInject`, a new open source injection technique that involves hooking an export function from a remote process to gain shellcode execution (avoiding the creation of a remote thread).

ThreadlessInject example detecting via the Windows System Module Remote Hooking rule

Conclusion

Until Microsoft provides vendors with kernel callbacks for security-relevant syscalls, Threat Intelligence ETW will remain the most robust visibility into in-memory threats on Windows. At Elastic, we're committed to putting that visibility to work for customers and optionally directly into their hands without any hidden tinkering assumptions.

Stay tuned for the call stack features in upcoming releases of Elastic Security.

Resources

Rules released with 8.11:

- AMSI or WLDP Bypass via Memory Patching
- Call Stack Spoofing via Synthetic Frames
- Suspicious Call Tracing for Windows Patching
- Memory Protection Modification of an Unsigned DLL
- Network Activity from a Stomped Module
- Potential Evasion via Invalid Code Signature
- Potential Injection via an Exception Handler
- Potential Injection via Asynchronous Procedure Call
- Potential Thread Call Stack Spoofing
- Remote Process Injection via Mapping
- Remote Process Manipulation by Suspicious Process
- Remote Thread Context Manipulation
- Suspicious Activity from a Control Panel Applet
- Suspicious API Call from a Stack Unwinder
- Suspicious API from an Unsigned Service DLL
- Suspicious Call Stack Tailing Bytes
- Suspicious Executable Heap Allocation
- Suspicious Executable Memory Permission Modification
- Suspicious Memory Protection Modification
- Suspicious Memory Write to a Remote Process
- Suspicious NTDLL Memory Write
- Suspicious Null Terminated Call Stack
- Suspicious Kernel32 Memory Protection
- Suspicious Remote Memory Allocation
- Suspicious Windows API Call from Virtual Disk or USB
- Suspicious Windows API Call via Direct Syscall
- Suspicious Windows API Call via ROP Gadgets
- Suspicious Windows API Proxy Call
- VirtualProtect API Call from an Unsigned DLL
- VirtualProtect Call via NtProtect
- VirtualProtect via Indirect Random Syscall
- VirtualProtect via ROP Gadgets
- Windows API via a Callback Function
- Windows System Module Remote Hooking

Share this article

f Facebook · X Twitter · in LinkedIn

Trademarks · Terms of Use · Privacy

© 2024. Elasticsearch B.V. All Rights Reserved.

This website and all contained content, software, discussion forums, products and services are intended for professional security use only. No consumer use of this website or its content is intended or desired. Elastic N.V. and its affiliates do not sell, provide, or make available any of its products or services to any consumer. Elastic N.V. and its affiliates do not sell personal information.

Elastic, Elasticsearch, Elastic Stack, Kibana, ECE, ECK and other Elastic products and marks are trademarks of the Elastic N.V. and affiliates. Apache Software Foundation, all other brand names, product names, or trademarks belong to their respective owners.